

SpotTune: Leveraging Transient Resources for Cost-efficient Hyper-parameter Tuning in the Public Cloud

Yan Li, Bo An, Junming Ma, Donggang Cao*, Yasha Wang, Hong Mei

Key Lab of High-Confidence Software Technology (Peking University), Ministry of Education, Beijing, China

{yan.l, anbo, mjm520, caodg, wangyasha, meih}@pku.edu.cn

Abstract—Hyper-parameter tuning (HPT) is crucial for many machine learning (ML) algorithms. But due to the large searching space, HPT is usually time-consuming and resource-intensive. Nowadays, many researchers use public cloud resources to train machine learning models, convenient yet expensive. How to speed up the HPT process while at the same time reduce cost is very important for cloud ML users. In this paper, we propose SpotTune, an approach that exploits transient revocable resources in the public cloud with some tailored strategies to do HPT in a parallel and cost-efficient manner. Orchestrating the HPT process upon transient servers, SpotTune uses two main techniques, fine-grained cost-aware resource provisioning, and ML training trend predicting, to reduce the monetary cost and runtime of HPT processes. Our evaluations show that SpotTune can reduce the cost by up to 90% and achieve a 16.61x performance-cost rate improvement.

Index Terms—cloud computing, transient resources, hyper-parameter tuning, machine learning

I. INTRODUCTION

Machine learning (ML) has become an important workload for both research and industry that tries to learn insight from the collected dataset. An ML model usually consists of a set of parameters, which can be updated during the training process. Yet there are also some non-trainable parameters that need to be determined by developers before training, e.g., number of hidden layers, learning rate, degree of regularization, kernel function of SVM, etc., namely hyper-parameters. In order to predict outcomes for new-coming data items more precisely, developers need to repeat the training processes with different hyper-parameter (HP) settings until getting the optimal model. This trial-and-error procedure is called hyper-parameter tuning (HPT).

Tuning hyper-parameters for an ML model is resource-intensive and expensive, requiring trying models with different configurations. The emerging public cloud has unlimited computation capacity, facilitating many developers to do HPT in parallel by renting a set of machines, which is convenient and fast, but very expensive. Many cloud vendors offer a great opportunity: transient availability of cheap but revocable resources [1], [2]. For example, AWS spot instances [3] and Google Cloud's preemptible resources [4] often provide users with machines at a 70%-80% discount compared with the regular price of the on-demand instances,

but with the risk that some of them might be taken away at any time.

Intuitively, using these dynamic resources to do HPT in parallel, i.e., launching a bunch of transient servers simultaneously, each of which tries a different hyper-parameter setting, could be both fast and cheap. But to use it robustly and efficiently, there are three key issues to consider.

Handle the interruption properly. Dynamic resources are cheap for a reason: they could be revoked at any time by the cloud vendors, which then interrupts the ML training. If no fault-tolerant strategies are applied, the intermediate training results would lose when encountering an interruption.

Cost-aware resources provisioning. There are hundreds of instance types in the public cloud dynamic infrastructure market, each of which has a different price and computation ability. Besides, the price of transient resources is variable. Facing such complex cloud markets, the user could be confused about how to provision resources to reduce the overall cost.

Make full use of the elasticity of the public cloud. The public cloud provides the ability of elasticity, i.e., acquiring/releasing resources on-demand and only paying for the used computation capacity. HPT is well-suited for this mechanism because, after the exhaustive searching of the hyper-parameters, only a small part of the models will be left. If the models initiated with bad hyper-parameters could be identified early, computation capacity could be released in time to lower the cost.

To tackle the above issues, this paper describes SpotTune, an orchestrating system that combines a fine-grained cost-aware resource provisioning strategy with an ML training trend predicting method to leverage transient cloud resources for HPT. SpotTune checkpoints the intermediate training data to the remote persistent storage when receiving the pre-delivered revocation notice, thus mitigating the risk of interruption, and its cost-efficient effect is powered by the following two principal techniques:

Fine-grained Cost-aware Resource Provisioning. SpotTune provisions resources for HPT jobs by computing the expected cost at a fine granularity. Via predicting spot instance revocation probability and online profiling HPT job performance, SpotTune computes the expected cost of a

*Corresponding author.

single iteration in the next hour and selects the instance that incurs the least cost to deploy HPT jobs.

Early-Shutdown based on Training Trend Predicting. SpotTune models the training process as a predictable curve, predicting the training trend of a model using the metrics (e.g., validation loss) collected during the training. Generally speaking, it can judge whether a model is promising or not using partial metrics thus could release resources running bad models in time. It just lets the models initiated with bad hyper-parameters stop before they take more resources to do the unnecessary computation, denoted as early-shutdown in this paper.

This paper makes the following four primary contributions.

- 1) It describes SpotTune, an orchestrating system that takes advantage of transient cloud resources to do HPT.
- 2) It introduces a novel transient resources provisioning approach by computing the expected cost at a fine granularity.
- 3) It proposes an enhanced spot instance revocation probability predicting method.
- 4) It introduces a new ML model training trend predicting method, modeling the training process as a staged predictable curve.

The reminder of this paper is organized as follows. Section II presents the background and motivation of this paper. Section III introduces the design of SpotTune. Section IV evaluates SpotTune. Section V narrates some discussions of our system. Section VI briefly introduces the related work. Section VII concludes this paper.

II. BACKGROUND AND MOTIVATION

This section briefly introduces the background, including existing transient cloud resource markets and their principal traits, hyper-parameter tuning, and the motivation of combining them together.

A. Transient Cloud Resources

Resources provisioned for business-critical workloads could be available at a discount in public pay-as-you-go clouds, creating an opportunity for other workloads. But, those resources may be taken back when needed by cloud vendors again. Taking Amazon AWS EC2 Spot Instance as an example, this subsection describes transient cloud resources and the opportunity they provide us.

Amazon Elastic Compute Cloud (Amazon EC2) [5] is a web service that provides secure, resizable compute capacity in the cloud. EC2 offers *on-demand instances*, which are reliable VMs charged at a per-second rate. Besides the reliable resources, EC2 also provides cheaper (usually 70%-80% lower than the on-demand resources) yet preemptible resources as *spot instances*. Spot instances could be revoked at any time, thus providing users a trade-off between reliability and cost savings.

There are several characteristics of the AWS EC2 spot instance that may affect the monetary cost and the possibility

of revocation. First, different instance types have different spot markets [1], [2]. Second, price fluctuations among different markets are barely correlated. Third, users could specify a *maximum price* when acquiring a spot instance. It could be considered as the maximum price the user would like to pay for it. Once the spot market price is over the user's *maximum price*, the instance would be revoked by AWS. Fourth, the user is charged at a per-second rate with the spot market price (not the maximum price) with an exception: users can get a full refund if the acquired instance is revoked in its first instance hour [6]. Besides, AWS provides a service that delivers termination notices [7], which are issued two minutes before the interruption. Otherwise, there is no refund. In fact, AWS used the term *bid* before 2017 and replaced it with *maximum price* after. But the refund mechanism still exists. Figure 1 shows an example of spot price changes over eleven days.

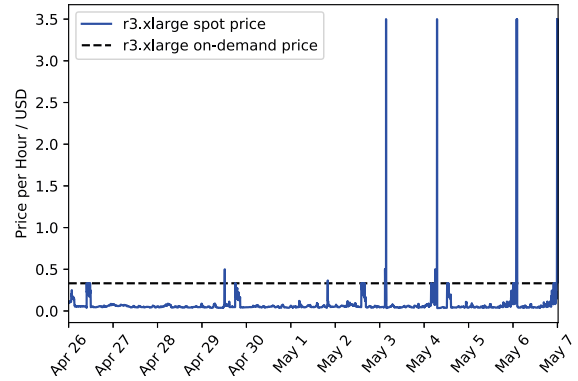


Fig. 1. AWS Spot Prices. Spot prices of instance type *r3.xlarge* across eleven days. The unchanging on-demand price of *r3.xlarge* is shown as well.

Collectively, there are two kinds of strategies to use EC2 spot instances. The first kind of strategy bid¹ **carefully** to minimize or mitigate the risks of being preempted. Zheng et al. [8] synthetically study the bidding strategies, deriving optimal bidding strategies for different job requirements. Sharma et al. [9] present the notion of server portfolios that is based on financial portfolio modeling for reducing the effect of preemption. Subramanya et al. [10] present SpotOn, automatically selecting a spot market and fault-tolerance mechanism to mitigate the impact of spot revocations. The other strategies usually just a little higher than the current spot market price, to maximize the odds of being revoked within the first hour, thus enjoying the refund bonus. This kind of bidding strategy is named as **aggressive** bidding. Harlap et al. [1] present Tributary, *borrowing* the to-be-preempted machines to satisfy a given service SLO with a lower cost.

¹Despite there is no longer the concept of *bid* any more now, the charging and refund mechanism still exists. And as these works are based on the concept of *bid*, we still use the term *bid* here.

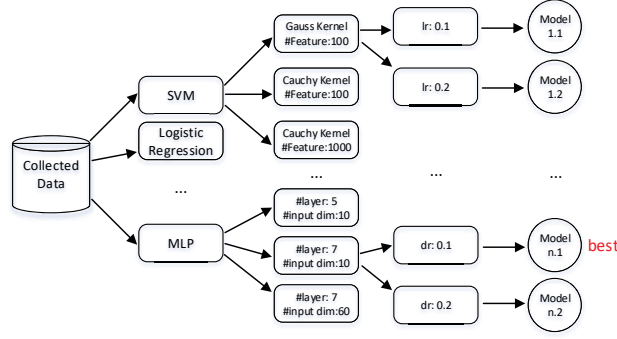


Fig. 2. An Example of **HPT**. After an exhaustive searching, only one optimal model is left. Here ‘lr’ denotes a learning rate parameter, ‘dr’ a dropout rate parameter, ‘#layer’ the number of layers in an MLP model, ‘#input dim’ the dimension of the input data, and ‘#Feature’ the number of features to use.

B. Hyper-parameter Tuning of ML Models

A machine learning model basically consists of a set of trainable parameters, which could be updated during the training process until the pre-defined model fits the training data. There are also some non-trainable parameters that should be determined by developers manually before training, namely hyper-parameters. HPT is, in fact, a process of selecting the best model among the HP space based on domain knowledge and trial-and-error. Figure 2 shows an example of HPT.

HPT can be done using various searching algorithms [11]–[17] over the space of the hyper-parameters, which means that the user launches multiple ML jobs, each performing full training using one hyper-parameter setting. Because the configuration space to search is usually large, this process is rather computationally expensive, especially in the context of public pay-as-you-go clouds.

Recent works [18], [19] have proved the predictability of ML model training, modeling the training process as a predictable curve. This mechanism could help lower the cost of HPT if combined with the elasticity and pay-as-you-go nature of the public cloud. That is, if bad models could be identified early by predicting the model quality with partially collected metrics (e.g., validation loss), the computation capacity could be released in time.

SpotTune follows this idea, modeling the ML training process as a staged predictable curve to early release machines running bad models.

C. Tuning with Transient Resources

Inspired by the properties of transient cloud resources, and the nature of HPT jobs, we present SpotTune, addressing the issues of the transient resources opportunity, the cloud elasticity and the ML training predictability to orchestrate HPT upon transient cloud resources in a parallel and cost-effective manner. Section III describes the design of SpotTune. To our knowledge, SpotTune is the first system leveraging transient resources in the public cloud to do HPT.

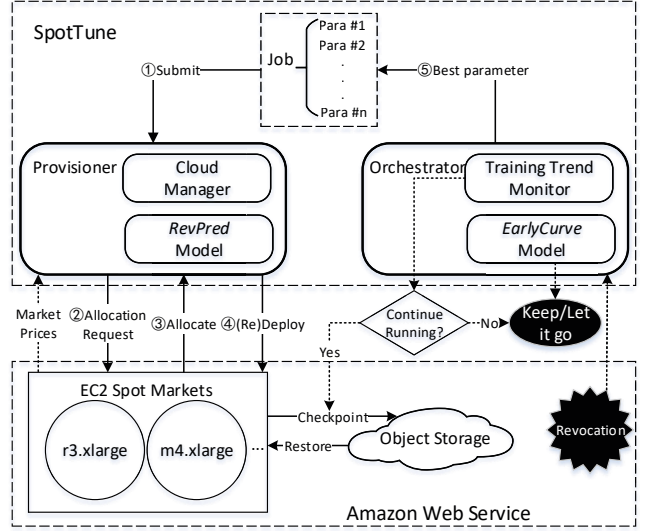


Fig. 3. SpotTune Overview. SpotTune consists of two important components, the Provisioner and the Orchestrator. The former is used to interact with the cloud to acquire resources and the later is responsible for orchestrating the training processes.

III. SPOTTUNE DESIGN

TABLE I
USER-SPECIFIED PARAMETERS IN SPOTTUNE

<i>metric</i>	the metric used to evaluate an ML model’s quality, e.g. validation loss, training MSE, etc.
<i>max_trial_steps</i>	the maximum steps the user would like to train the ML model
θ	the early-shutdown rate to predict the final metric value using partial collected metric data
<i>mcnt</i>	the number of models the user would like select from all the HPs

In this section, we detail the design of SpotTune. SpotTune has two main concerns: **i)** finishing the HPT process with as low cost as possible, and **ii)** selecting (near) optimal models from all the HPs after a fixed number of iterating steps.

A. Overview

Figure 3 depicts a high-level view of SpotTune and its operational environment. In a word, SpotTune takes advantage of cheap transient resources and the elasticity of the public cloud to do HPT in a cost-efficient manner. Noted that except for the ML model, all the HPs, and the training data, the user additionally needs to specify four other important parameters as described in Table I, which could be assigned according to the real needs.

Provisioner in SpotTune interacts with AWS to request and acquire resources. By computing the expected cost according to the output of RevPred and the online-updated performance data, Provisioner chooses the proper instances to deploy the HPT jobs.

Algorithm 1 SpotTune Workflow

Input: *HPs*: hyper-parameter settings, *spotPool*: spot instance pool, *max_trial_steps*, θ , *mcnt*

```
1: function GETBESTINST(t, hp, M)
2:   sCost  $\leftarrow$  new Map()
3:   for inst  $\in$  spotPool do
4:     price  $\leftarrow$  RANDOM(0.00001, 0.2) + pricecurrent
5:     p  $\leftarrow$  REVPREDMODEL(inst, t, price)
6:     sCost[inst]  $\leftarrow$  M[inst][hp]  $\times$  (1 - p)  $\times$   $\overline{price}$ 
7:   end for
8:   return argmin(sCost)
9: end function
10:
11: assign  $\leftarrow$  new Map(default value=null)
12: M  $\leftarrow$  new Map(default value= $C_0 \times inst.CPU$ )
13: waiting  $\leftarrow$   $\emptyset$ , finished  $\leftarrow$   $\emptyset$ 
14:
15: while True do
16:   toQuery  $\leftarrow$  (HPs - finished)
17:   if toQuery ==  $\emptyset$  then
18:     break
19:   end if
20:   for hp  $\in$  toQuery do
21:     VM  $\leftarrow$  assign[hp]
22:     if VM == null then
23:       waiting.add(hp)
24:     else if receive the revocation notice of VM then
25:       checkpoint result of hp to S3
26:       waiting.add(hp)
27:     else if hp has ran  $\theta \times max\_trial\_steps$  then
28:       checkpoint result of hp to S3
29:       shutdown VM
30:       finished.add(hp)
31:     else if hp ran on VM for > one hour then
32:       checkpoint result of hp to S3
33:       shutdown VM
34:       waiting.add(hp)
35:     end if
36:     UPDATEMETRICS(M, inst(VM), hp)
37:   end for
38:   for hp  $\in$  waiting do
39:     inst  $\leftarrow$  GETBESTINST(tnow)
40:     VM  $\leftarrow$  REQUESTAWS(inst)
41:     deploy hp on VM
42:     waiting.remove(hp)
43:     assign[hp]  $\leftarrow$  VM
44:   end for
45:   SLEEP(10 seconds)
46: end while
47:
48: metrics  $\leftarrow$  new Map()
49: for hp  $\in$  finished do
50:   metrics[hp]  $\leftarrow$  EARLYCURVE(hp, max_trial_steps)
51: end for
52: SORT(metrics)
53: continue training top-mcnt models from checkpoints
```

Orchestrator is responsible for monitoring the training trend and predicting the final metric of every HPT job using partially collected metrics (e.g., validation loss) during training. And when revocation happens, Orchestrator notifies the VM to checkpoint the intermediate training results to the remote object storage S3.

Algorithm 1 describes the entire workflow of SpotTune to run the HPT jobs of a machine learning algorithm.

The expected cost in the next hour for all the instances could be calculated using the following equation:

$$\begin{aligned}\mathbb{E}[eCost] &= [(1-p) \cdot \overline{price} + p \cdot 0] \cdot 1 \text{ hour} \\ &= (1-p) \cdot \overline{price} \cdot 1 \text{ hour}\end{aligned}\quad (1)$$

where p is the revocation probability of current instance predicted by **RevPredModel** which we will introduce the detailed mechanism in the next subsection, and $\overline{price}$ is the average price of this instance in the last hour. Then function **getBestInst** returns the instance that has the lowest **step cost** (\$ per step) in the next hour, which is the monetary cost of running a single iteration, defined as the following equation:

$$\begin{aligned}\mathbb{E}[sCost] &= \mathbb{E}\left[\frac{M[inst][hp] \cdot eCost}{1 \text{ hour}}\right] \\ &= \frac{M[inst][hp] \cdot \mathbb{E}[eCost]}{1 \text{ hour}} \\ &= \frac{M[inst][hp] \cdot (1-p) \cdot \overline{price} \cdot 1 \text{ hour}}{1 \text{ hour}} \\ &= M[inst][hp] \cdot (1-p) \cdot \overline{price}\end{aligned}\quad (2)$$

where M is a two-dimensional matrix recording the performance of running different HPs on different instances. For example, $M[r3.xlarge][hp_1]$ (seconds per step) represents the speed of running HPT job hp_1 on instance $r3.xlarge$. M is initiated according to the number of CPU cores of each instance. During the HPT process, M would be updated in an online manner according to the latest runs as shown in line 36 of Algorithm 1. This performance modeling method is practical because the computation pattern of a concrete HPT job is steady across different iterations, which we will demonstrate in our experiments (Subsection IV-A5).

Then as the line 16-46 of Algorithm 1 shows, SpotTune loops to wait for three events: **1)** when receiving the pre-delivered revocation notice from AWS, SpotTune checkpoints its intermediate training results to S3 and arranges a re-deployment; **2)** when an HPT job finishes, SpotTune checkpoints its intermediate training results to S3 and marks it as *finished*; **3)** when an HPT job has run on an instance for more than one hour, SpotTune checkpoints its intermediate training results and shutdown the VM to arrange for a re-deployment.

As we analyze in Section I, HPT usually only leaves a small portion of models after trying them all, which means that a large amount of computation is meaningless. So SpotTune would not finish all the training steps, it would stop at $\theta \times max_trial_steps$ and predict the final quality (e.g., validation loss) of every model. After that, it would continue the training of the top-*mcnt* models from the checkpoints. Line 48-53 reveals this procedure.

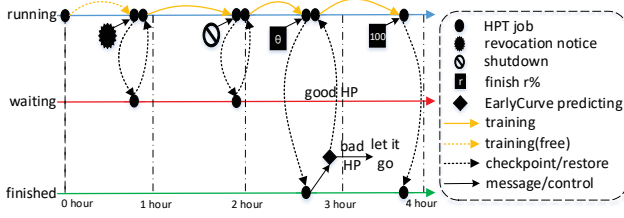


Fig. 4. An Example of the Lifetime of an HPT Job Using SpotTune.

Figure 4 depicts a typical example of the lifetime of an HPT job in SpotTune. As the demo figure shows, the VM that the model is deployed on is revoked in less than one hour so the initial tens of minutes are free. Then SpotTune forwardly shuts down the model training process when it has not been revoked for more than one hour. And when the model has been trained for $\theta \times \text{max_trial_steps}$, EarlyCurve determines whether to finish the entire training.

According to Algorithm 1, the functionality of SpotTune is guaranteed by two important prediction models, **RevPred** (line 5th) and **EarlyCurve** (line 50th). We show the details of these two models in the next two subsections.

B. Spot Market Price Modeling

As Equation 2 shows, the revocation probability is one of the factors that SpotTune uses to calculate the *step cost*. Give an instance type I , a *maximum price* b and a timestamp t , **RevPred** could output the revocation probability $P(I, b, t)$ of this instance in the next hour. The higher it is, the more possible SpotTune may select it. Similar to Tributary [1], which is the state-of-the-art spot instance revocation probability predicting algorithm, RevPred uses an *LSTM*-based deep neural network model structure to extract the history price knowledge. But in RevPred, there are two main differences: **i)** the *LSTM* structure in RevPred only handles partial input data; **ii)** RevPred adopts a different data pre-processing approach to generate the *maximum price*. For each individual spot market, an independent model is trained offline using the history spot price dataset.

The input data of our model is separated into two parts. The first part is the history prices across the past 59 minutes, with each price record containing the following six engineered features: (1) current spot market price; (2) average spot market price; (3) the number of price changes in the past hour; (4) the time duration since the current spot market price is set; (5) whether the time is in the workdays or not; (6) current hour of the day. The second part is the present price record, with the above six features and the *maximum price*.

In Tributary, the *maximum price* is randomly generated when training the LSTM model, which is slightly higher than the present spot market price by a random delta in a pre-defined interval $([0.00001, 0.2])$ [1]. While in RevPred, we argue that the delta should be determined differently in training and inference. When training the model, the *maximum price* $b(I, t)$ at time t is determined by the market

price fluctuation as Algorithm 2, aiming at calculating the average variation of instance I 's history prices (removing the smallest 20% and the largest 20%) in the previous 1 hours as the deltas in the training data. This idea comes from the essence of *active learning*, which suggests that the data points locating on the classification border would help improve the model most significantly. And intuitively, the average variation of prices plus the current price would be close to the border that separates *revoked* and *not revoked*. If the spot market price grows over the maximum price in the following one hour, we label this record as *True*. While using the trained model for inference, RevPred randomly generates the maximum price $b(I, t)$ as Tributary does.

Algorithm 2 Training Data Pre-processing

Input: *prices*: the history price data of spot instances used to train RevPred, I : spot instance type, t : timestamp

- 1: $\text{deltas} \leftarrow \text{new List}()$
- 2: **for** $(t - 1 \text{ hour}) < \tau < t$ **do**
- 3: $\text{deltas.append}(|\text{prices}[I][\tau] - \text{prices}[I][\tau - 1]|)$
- 4: **end for**
- 5: $\text{SORT}(\text{deltas})$
- 6: $L \leftarrow \text{LENGTH}(\text{deltas}), b \leftarrow 0$
- 7: **for** $0.2 \times L < i < 0.8 \times L$ **do**
- 8: $b \leftarrow b + \text{deltas}[i]$
- 9: **end for**
- 10: $b \leftarrow \text{prices}[I][t] + b/(0.6 \times L)$

The 59 price records in the past hour are fed into a three-tier *LSTM* structure to retrieve the history spot market information, coming out as an embedding feature vector. The present price record is converted to an embedding presentation too through three sequential fully connected layers. Then these two feature vectors are concatenated together to come up with a probability-like result.

The spot market price data could be very skewed. We use two techniques to mitigate the data-imbalance issue. First, we carefully design the loss function by assigning different weights for different classes. Let ϕ_+ and ϕ_- denote the percentage of the positive samples (marked as *True*) and negative samples in the training set, then we assign ϕ_- and ϕ_+ as the positive and negative class weights. Second, we do not use the output of the prediction model as the probability directly. Denoting the output of the model for I with price b at time t as $\hat{P}(I, b, t)$, we compute the preemption probability $P(I, b, t)$ as:

$$\frac{P(I, b, t)}{1 - P(I, b, t)} = \frac{\hat{P}(I, b, t) \cdot \phi_-}{[1 - \hat{P}(I, b, t)] \cdot \phi_+} \quad (3)$$

Once a RevPred model is trained using the history price data of an instance I , it could predict the probability of preemption $P(I, b, t)$ in the following hour given the timestamp and the *maximum price*.

C. ML Training Trend Modeling

EarlyCurve uses partial metrics to predict the final metric of an ML model, thus to save computation capacity by

TABLE II

SUMMARY OF ML ALGORITHMS, DATASETS, OPTIMIZERS, METRICS AND HYPER-PARAMETER SETTINGS WE USED FOR TESTING. NOTATIONS: *bs*: BATCH SIZE, *lr*: LEARNING RATE, *dr*: DECAY RATE, *ds*: DECAY STEPS, *kernel*: SVM KERNEL FUNCTION, *nt*: #TREES IN GBTR, *version*: RESNET VERSION, *depth*: MAX TREE DEPTH IN GBTR OR NUMBER OF CORE CONVOLUTIONAL LAYERS IN RESNET, *de*: DECAY EPOCHS

Algorithm	Dataset	Optimizer	Metric	HP settings
Logistic Regression(LoR)	Epsilon [20]	Gradient Descent	Cross Entropy	$bs \in \{128, 64\}$ $lr \in \{1e-2, 1e-3\}$ $dr \in \{1.0, 0.95\}$ $ds \in \{1000, 2000\}$
Support Vector Machine(SVM)	Synthetic	Gradient Descent	Hinge Loss	$bs \in \{128, 64\}$ $lr \in \{1e-2, 1e-3\}$ $dr \in \{1.0, 0.95\}$ $kernel \in \{RBF, Linear\}$
GBT Regression(GBTR)	Synthetic	Gradient Boosting	MSE	$bs \in \{128, 64\}$ $lr \in \{1e-1, 1e-2\}$ $nt \in \{10, 15\}$ $depth \in \{5, 8\}$
Linear Regression(LiR)	YearPredictionMSD [21]	Gradient Descent	MSE	$bs \in \{128, 64\}$ $lr \in \{1e-2, 1e-3\}$ $dr \in \{1.0, 0.95\}$ $ds \in \{1000, 2000\}$
AlexNet(AlexNet)	Cifar10 [22]	Adam [23]	Cross Entropy	$bs \in \{128, 64\}$ $lr \in \{1e-1, 1e-2\}$ $dr \in \{1.0, 0.95\}$ $de \in \{40, 60\}$
Residual Neural Network(ResNet)	Cifar10	Adam	Cross Entropy	$bs \in \{32, 64\}$ $version \in \{1, 2\}$ $depth \in \{20, 29\}$ $de \in \{40, 60\}$

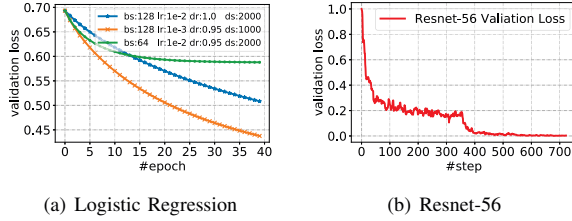


Fig. 5. (a): Logistic Regression validation loss(validating every epoch) curves on Epsilon dataset using three different hyper-parameter settings, where *bs* denotes the batch size, *lr* the initial learning rate, *dr* the decay rate of the learning rate and *ds* the number of steps between two learning-rate-decay. (b): Resnet-56 validation loss on CIFAR10 dataset. This model applies a learning rate that decays periodically such that the validation curve is separated to two different stages.

shutting down bad models early. The user could indicate different metrics for different problems. For example, training/validating cross-entropy for classification problems and training/validating MSE (mean squared error) for regression problems. The algorithms and metrics used in our experiments are shown in Table II. ML algorithms listed in Table II use gradient-based optimization approaches to update trainable parameters, which converges (assume the loss function f is convex, differentiable, and ∇f is Lipschitz continuous) at a rate of $O(\frac{1}{k})$ in terms of the number of steps k (with optimized versions of gradient descent/boosting, the convergence rate could be improved to $O(\frac{1}{k^2})$) [18]. Figure 5(a) shows an example of different shapes of validation loss curves using different HP settings. EarlyCurve uses the following model (algorithms optimized with optimizers that converge at a different rate are discussed in Section V) to fit the validation metrics during the training process:

$$\hat{\mathcal{L}}_k = \sum_{i=1}^{ST} \left(\frac{1}{\alpha_{i0} \cdot k^2 + \alpha_{i1} \cdot k + \alpha_{i2}} + \alpha_{i3} \right) \cdot \text{sign}(k, l_i, r_i) \quad (4)$$

where $\hat{\mathcal{L}}_k$ denotes the model's predicted metric at step (epoch) k , α_{i0} , α_{i1} , α_{i2} and α_{i3} are non-negative coefficients, and function sign is defined as:

$$\text{sign}(k, l_i, r_i) = \begin{cases} 1, & l_i \leq k \leq r_i \\ 0, & k < l_i \text{ or } k > r_i \end{cases} \quad (5)$$

Where l_i and r_i denote the left and right bound of an interval, which satisfies the following condition at time T :

$$\bigcup_{i=1}^T [l_i, r_i] = [0, T), \text{ and, } \bigcap_i [l_i, r_i] = \emptyset \quad (6)$$

Unlike prior works [18], [19], Equation 4 takes the multi-stage issue like Figure 5(b) into consideration. That is, in the training process of an ML model, the learning rate could be different at different stages (ST denotes the #stage) such that the metrics could vary sharply at some certain points, so we model the training process as a piecewise function. EarlyCurve fits the training curve in an online manner, determining when to separate the curve to a new stage according to the following criteria:

$$\zeta_i = \frac{|\mathcal{L}_i - \mathcal{L}_{i-1}|}{\mathcal{L}_{i-1}} > \xi, \quad (7)$$

$$\zeta_j < \varepsilon, j \in \{i-1, \dots, i-5\}$$

where $\mathcal{L}_i$ denotes the to-be-evaluated metric at step i , ζ_i the changing rate of $\mathcal{L}$ at step i , ξ the pre-defined threshold of metric's changing rate (0.5 in our experiment), and ε the threshold of last five steps' change rate (0.01 in our experiment). Equation 7 is heuristic: if the changing rate of a model's metric is suddenly high after a steady period, it could be considered to be moving to a new stage.

The least number of steps used to predict the final metric is controlled by the parameter θ . For example, if $\theta=0.8$ and $\text{max_trial_steps}=10000$, EarlyCurve would predict the final metric after running 8000 steps. Through our experiments, we observe that the starting 70% (see Section IV for more detail) points of all the training steps could support a

TABLE III
EXPERIMENTAL INSTANCE CONFIGURATIONS

Instance Type	CPUs	Memory(GB)	On-demand Price(USD/hr)
<i>r4.large</i>	2	15.25	0.133
<i>r3.xlarge</i>	4	30	0.33
<i>r4.xlarge</i>	4	30.5	0.266
<i>m4.2xlarge</i>	8	32	0.4
<i>r4.2xlarge</i>	8	61	0.532
<i>m4.4xlarge</i>	16	64	0.8

well-behaved fitting model, mostly predicting a nearly 70% accuracy and 100% top-3 accuracy. We measure SpotTune’s sensitivity against θ in Section IV.

One extra special case should be mentioned, which is that the model comes to convergence before $\theta \cdot \max_trial_steps$ steps. After that, the metric curve becomes a plateau, where training is no longer meaningful anymore. In this case, we stop the iteration and treat this model as finished.

EarlyCurve works as follows: As the training progress goes on, the SpotTune Orchestrator keeps monitoring the metric data point $(k, \mathcal{L})$. If the number of already run steps is above $\theta \times \max_trial_steps$, EarlyCurve then uses a linear regression solver [24] to find the best coefficients that fit the collected metric data points and predict the final metric value.

IV. EVALUATION

We evaluate SpotTune from four aspects. First, we evaluate the cost-saving effects and performance by simulating the HPT process using real-world ML benchmarks and EC2 spot market history price data. Second, we try to explain why SpotTune is cost-effective. Third, we analyze the performance of RevPred and EarlyCurve compared with previous works. Finally, we discuss the overhead of our system.

A. Experimental Setup

1) **Spot Market Price Data:** We use the open-source dataset *AWS Spot Pricing Market* [25] publicized on Kaggle. It collects the spot prices across eleven regions from 2017-04-26 to 2017-05-08. Theoretically, any price datasets could be adopted in the training of RevPred. We use the *us-east-1.csv* sub-dataset, covering 68 types of instances. For simplicity, we choose six representative types as the experimental spot instance pool. The configurations and on-demand prices of these five instances are shown in Table III. Noted that this dataset is rather sparse, which means that the time intervals between adjacent records are non-uniform. So we preprocess the dataset by interpolating values between records, making the timestamp interval between adjacent records fixed at 1 minute.

2) **ML benchmarks:** We test our system with some popular ML algorithms, including (i) classification algorithms: Logistic Regression, SVM, and CNN-based neural network schema AlexNet and ResNet; (ii) regression algorithms: Gradient Boosting Tree Regression and Linear Regression.

For each algorithm, we try a set of different hyper-parameter settings as shown in Table II. For example, SVM with different kernels, ResNet with different numbers of core convolutional layers.

3) **Datasets:** To evaluate SpotTune’s general ability in the real world, we use multiple datasets we collected from various online sources with some modifications and our own dataset. The datasets span a variety of types (including images [22], [26], binary-classification bench [20], audio meta-features [21], and our random-generated synthetic dataset), as shown in Table II. In our experiments, all the training datasets are serialized to python pickle objects, summing up to over 100GB.

4) **Baseline:** The baseline we compare SpotTune with is running HPT on a single spot instance. We assume the *maximum price* of each used single-spot instance is much higher than its market price such that it would not be revoked. Many previous works adopt similar strategies to mitigate the risk of revocation [10]. There are two kinds of representative spot instances in our experiment, the *cheapest* (*r4.large*) and the *fastest* (*m4.4xlarge*), denoted as *Single-Spot Tune (Cheapest)* and *Single-Spot Tune (Fastest)* in Figure 7. There are two version SpotTune in our experiment, with θ set as 0.7 and 1.0 respectively. As the Top-3 accuracy reaches 100% when $\theta \geq 0.7$, we treat 0.7 as the minimum reliable θ by default. If the user indicates a θ of less than 0.7, he might take the misprediction risk of EarlyCurve.

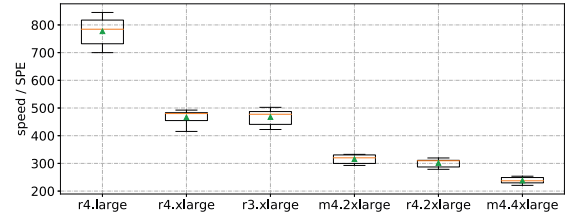


Fig. 6. Performance Profiling Example.

5) **Performance Profiling:** As described in Algorithm 1, we profile the performance of running different HPT jobs on different instances in an online manner. To prove the feasibility of this profiling method, we pre-run all the models in Table II on all the instances in Table III for N steps (epochs) and record the time consumption of each step (epoch) *seconds/batch(SPB)* (or *seconds/epoch(SPE)*) as $M[inst][HP]$. We observe that the coefficient of variation (COV) of $M[inst][HP]$ across different steps is small enough (less than 0.1), which demonstrates the online profiling method is practical. Figure 6 shows an example of the performance metrics of different types of spot instances in our experiment, whose vertical axis represents the speed (SPE) of training the ResNet model. And as the horizontal axis goes right, the price of the instance is increasing. An interesting observation goes to that as the prices increases, the performance does not get better accordingly all the time, at least not in a linear manner.

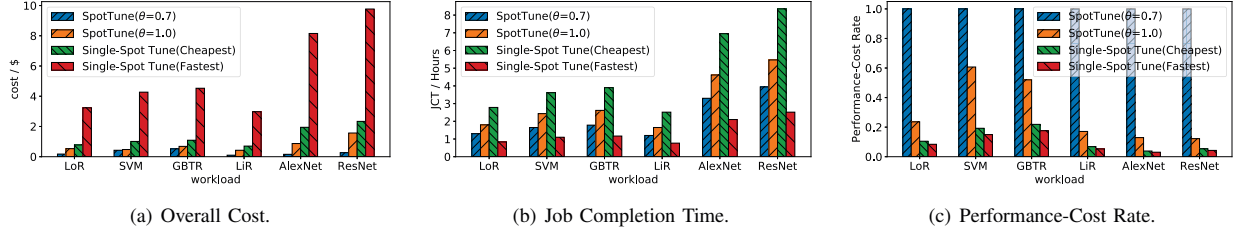


Fig. 7. Cost/Performance Comparison between SpotTune and Two Baselines.

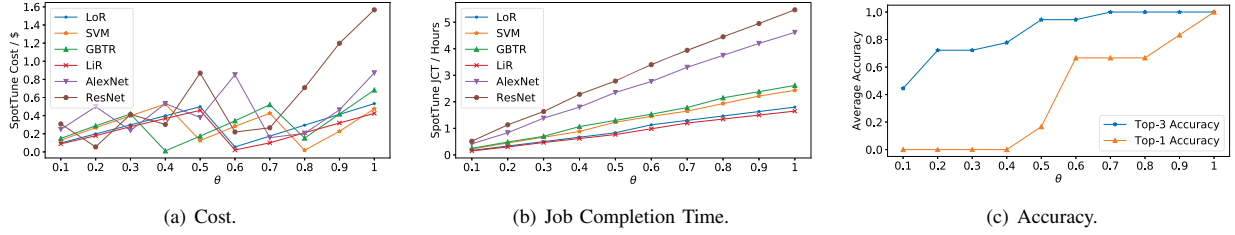


Fig. 8. SpotTune's Sensitivity against θ

B. Cost and Performance Evaluation

1) **Overall Cost and Performance:** To evaluate the cost-saving effect and performance of SpotTune, we simulate the HPT process of all the benchmark workloads shown in Table II. Figure 7 shows the simulation results. When θ is set 1.0, the EarlyCurve component does not work, so the two baselines could be considered as $\theta=1.0$ as well. It should be mentioned again that we assume the *maximum price* of the single-spot instance approach is high enough to avoid the preemption.

In Figure 7(a) we show the overall cost yielded by four approaches. Averagely speaking, the overall cost of SpotTune ($\theta=0.7$) is the lowest across all the six simulating workloads. SpotTune ($\theta=1.0$) saves 41.5% and 86.04% compared with the cheapest and the fastest standalone spot instance respectively. Besides, SpotTune ($\theta=0.7$) saves 57.16% compared with SpotTune ($\theta=1.0$) because of the 30% less running steps. Finally, SpotTune ($\theta=0.7$) could save up to 75.64% and 94.18% compared with Single-Spot Tune (Cheapest) and Single-Spot Tune (Fastest) because of the refund bonus and the less trying steps.

Figure 7(b) illustrates the HPT job completion time (JCT) of four approaches, where JCT is defined as the time span from the HPT job submission to selecting the best model(s). JCT composes of two main parts: training and checkpoint-restore. We will evaluate the checkpoint-restore time percentage in Subsection IV-F. Intuitively, the JCT of SpotTune is between two baselines because it uses a mixture of all the instances. SpotTune ($\theta=0.7$) can achieve 64.59% of the fastest instance's performance and have a 2.14x boosting compared with the cheapest instance.

Figure 7(c) depicts the performance-cost rate (PCR) of four approaches, which is calculated by $\alpha/(JCT \cdot cost)$ where α is a constant, revealing each approach's perfor-

mance of a unit cost. The less the JCT and the cost are, the higher the performance-cost rate is. We normalize the PCR data for each benchmark, fixing the SpotTune ($\theta=0.7$) PCR as 1. With all running the same number of steps, SpotTune ($\theta=1.0$) has a 2.65x and 3.36x PCR compared with Single-Spot Tune (Cheapest) and Single-Spot Tune (Fastest). With θ set as 0.7, SpotTune ($\theta=0.7$) has a 13.11x and 16.61x PCR compared with the two baselines.

2) **Sensitivity against θ :** SpotTune has one parameter θ that needs to be determined by the user, so we evaluate the sensitivity of SpotTune against θ . Figure 8 shows the results. θ reflects the user's attitude toward early-shutdown. If θ is small (e.g., 0.1), it reflects that the user is radical so as to select the *best* model more quickly yet might take the risk of misprediction. Otherwise, the user may want to do the HPT in a safe and sound manner. We let the θ vary from 0.1 to 1.0 with an interval of 0.1.

Figure 8(a) depicts the cost yielded by SpotTune under different θ and benchmarks. Generally speaking, the overall cost is proportional to θ . But sometimes the higher θ yields lower cost instead. Taking the SVM benchmark as an example, the overall cost when $\theta=0.8$ is less than when $\theta=0.7$. This is because some instances are terminated by SpotTune when the already-run steps achieving $\theta \times max_trial_steps$ when θ is lower but are revoked by AWS when θ is higher thus incurring more refund. The JCT of SpotTune varies near-linear to θ as illustrated in Figure 8(b). Figure 8(c) shows the accuracy/top-3 accuracy of the final selected model, i.e., the accuracy of EarlyCurve. When θ comes to 0.7 or higher, EarlyCurve can predict a 100% top-3 accuracy.

According to SpotTune's sensitivity against θ , the user could indicate θ to meet his demand. For example, if the user wants to select the *near-best* model as quickly as possible, he could indicate the θ as 0.2-0.4, getting a 70%-80% top-

3 accuracy. If the user persists to get the highest top-1 accuracy, θ should be assigned as 1.0, i.e., without early-shutdown.

C. Why SpotTune is the Cheapest

Figure 7 shows SpotTune is faster yet cheaper than the single cheapest spot instance. We find that there are two main reasons: **i)** the instance with the lowest price may not be the cheapest instance to run the entire workload; **ii)** the minimization of Equation 2 relies on three factors: performance average price, and revocation probability of an instance. As we always re-schedule the running instance every one hour, the instance with higher revocation probability would be favored by SpotTune. We evaluate the numeric benefits brought by the refunded resources. Figure 9 shows the contribution/cost-saving percentage of the refunded (free) resources. As shown in Figure 9 ($\theta = 0.7$), the refunded resources contribute an average of 77.5% steps. In other words, taking the Logistic Regression benchmark in which we set the *max_trial_steps* as 1000 as an example, there are $1000 \times 0.7 \times 75\% = 525$ steps are run freely. The refund is an important reason that SpotTune is faster yet cheaper than *Single-Spot Tune (cheapest)*.

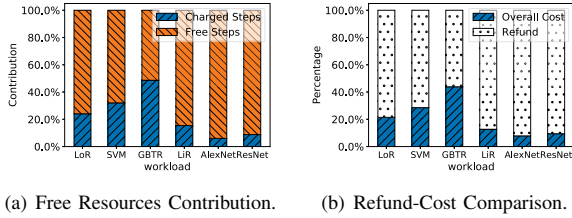


Fig. 9. Contribution/Cost-saving Percentage of the Refunded Resources.

D. RevPred Performance

This section evaluates the accuracy and effect of RevPred, which is described in Section III-B. The recent Tributary [1] system used the machine learning inference models trained with historical spot market data with engineered features and random-generated *maximum price*. RevPred in SpotTune improves the prediction accuracy by **i)** separating the input data into history data and present data and **ii)** generating *maximum price* according to the history price fluctuation when training the model, which we argue are the two reasons that RevPred outperforms Tributary. Since Tributary does not open-source its codes, we try to re-implement its prediction model from scratch, which is named as Tributary predict in our experiments. Figure 10 shows the accuracy and F1 scores for three prediction models, RevPred, Tributary Predict, and Logistic Regression. These models are trained on spot market price data from 04/26/17-05/04/17 and are evaluated on data from 05/05/17-05/07/17 for instance types in Table III.

The output of the prediction models is whether the instance specified with a price will be preempted within an hour. Accuracy scores are calculated by the *#samples*

predicted correctly divided by the total *#samples*. F1 score is a synthetic accuracy measurement when the dataset is skewed. RevPred provides the best accuracy and the best F1 score. RevPred increases accuracy by 20.32% and F1 score by 34.03% compared with Tributary Predict.

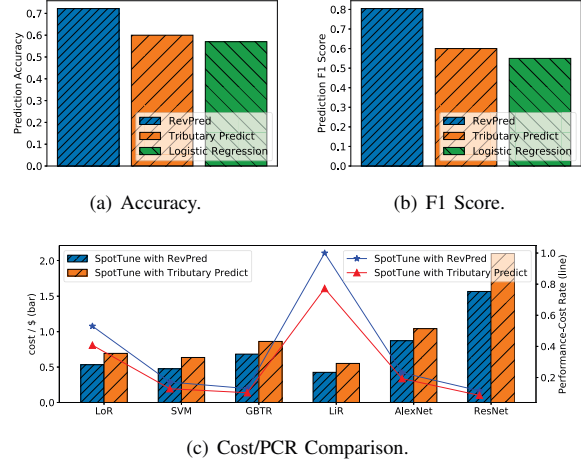


Fig. 10. Comparison between RevPred and Baselines

The increased accuracy and F1 scores are transferred to the effectiveness of SpotTune because the accurate predicting enables SpotTune to provision instances with lower *step cost* more precisely. We integrate two spot instance revocation probability predictors, RevPred and the Tributary predict, with SpotTune to compare the cost and normalized PCR. As Figure 10(c) shows, when using RevPred, SpotTune yields about 25% less cost and 24% more PCR compared to using the Tributary predictor.

E. Training Trend Predicting Effect

We compare EarlyCurve with previous training trending predicting method SLAQ [19] using the benchmark training metrics data with θ fixed as 0.7. Figure 11(a) shows an example of fitting curves using the two methods. As the baseline uses one-stage curve fitting, which does not take the multi-stage learning rate issue into consideration, its prediction error is significantly higher than EarlyCurve. Noted that if the learning rate of a model is not changing periodically, EarlyCurve and SLAQ would exhibit the same effect. Figure 11(b) shows the prediction error of the two

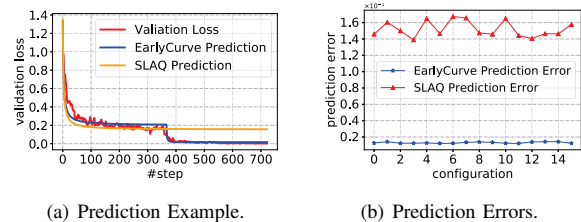


Fig. 11. Training Trend Prediction Effect.

methods on all 16 configurations of ResNet. The prediction accuracy of RevPred is shown in Figure 8(c).

F. Checkpoint-Restore Overhead

Checkpointing models to the remote object storage is the main overhead in SpotTune. We use the s3fs tool [27] to mount the remote object storage AWS S3 to a local directory on the machine so as to checkpoint the model conveniently. Besides, all the training data is restored on S3 before training as aforementioned, such that a new-started instance could instantly start to training without doing too much preparing work. Theoretically the maximum model size that could be uploaded to the remote storage would be the speed of checkpointing times 120 seconds because the notice is delivered two minutes before the real revocation as aforementioned.

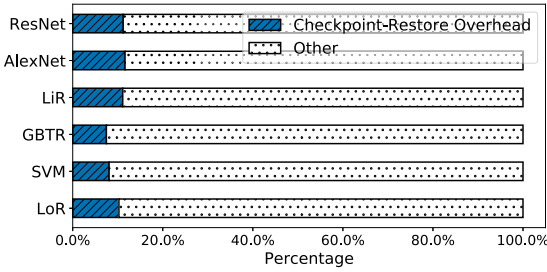


Fig. 12. Checkpoint-Restore Time Percentage.

We measure the speed of checkpointing models to AWS S3 [28] using Tensorflow on different types of instances. We observe that checkpointing is a CPU-bound process. The instance with the most cores, *m4.4xlarge*, has the highest checkpointing speed 134.22MB/s and the largest max-model-size 15.73GB. For comparison we use the *t2.micro* instance, with 1 CPU core and 1GB memory, as the tested, achieving a 62.83MB/s speed and a 7.36GB model size upper bound. If the model size is over the maximum model size an instance supports, SpotTune could use periodically checkpointing or prediction-based checkpointing to handle it, which we leave for future work.

Figure 12 shows the percentage of time fraction checkpoint-restore takes compared with the training time, less than 10% on average. Compared with the training time and the cost-saving effect, we argue the checkpointing is worthy.

V. DISCUSSION

A. Spot Instance Pool

According to Equation 2 and our analysis in Subsection IV-C, SpotTune may favor the unsteady spot markets such that could acquire more refund using our resource provisioning approach. But in the real world, spot markets exhibit various pricing patterns, among which some are very stable such that may not be preempted frequently. In this case, SpotTune degenerates to a trivial scenario without benefiting from the refund, i.e., the p in Equation 2 tends to zero.

In this scenario, SpotTune would choose the instance with the lowest *step cost*, without considering the revocation too much.

B. ML Model Convergence Rate

EarlyCurve in SpotTune models the ML training curve as a process converging at a speed of $O(\frac{1}{k})$ or $O(\frac{1}{k^2})$, namely sublinear. This magnitude of convergence rate is well-suited for gradient-descent-based ML optimization algorithms. But there are also some algorithms with linear or superlinear convergence rates, which converge at a rate of $O(\mu^k)$. For example, L-BFGS which is a widely used Quasi-Newton Method has a superlinear convergence rate which is between linear and quadratic [19]. Experimental ML benchmarks in this paper all use gradient descent as the optimization algorithms. If the user uses SpotTune to select the best hyper-parameter setting of a superlinear converging model, a different curve-fitting model should be applied, which we will investigate in future work.

C. Non-converging ML Models

SpotTune's ML training trend prediction is based on the convergence property of the underlying optimizers with the metrics history data. Loss functions of some models initiated with bad hyper-parameters may not monotonically decrease, causing SpotTune to mispredict the training trend. In this case, the user may assign θ as 1 and indicate a proper metric representing the model's quality under a concrete hyper-parameter setting.

VI. RELATED WORK

A. Hyper-Parameter Tuning

Many previous works [11]–[17] focus on improving the hyper-parameter search algorithms to reduce the times of attempting steps, while SpotTune looks at the underlying system/resources perspective to HPT. These algorithms could be integrated with SpotTune.

B. Leveraging Transient Resources

Many batch computing and elastic web service systems [1], [2], [10], [29]–[31] take advantage of transient resources to achieve a lower cost. SpotTune runs a new workload, HPT, on transient resources, which is different from them in the basic motivation. And SpotTune's resources acquisition mechanism is better than prior works such as Tributary [1], which could also be applied to enhance the batch computing or elastic web service systems with modifications.

Also, there are some works leveraging transient cloud resources for ML use at scale. Proteus [2] focuses on building a parameter server architecture using transient cloud resources for training an ML model in a distributed manner. SpotTune is different from it because it solves the problem of training models with different HP settings simultaneously rather than running a single model. MARK [32] takes advantage of transient resources for cost benefits, but mainly focusing on the process of the model serving stage.

C. Spot Market Price Prediction

Prior works [1], [2], [10] use different methods, including simple heuristic models and machine learning models, to predict the spot market pricing trend of AWS EC2. They either predict the future price of spot instances or the probability of revocation. The trend of the spot market price could be used to aid in the pro-active checkpointing or cost-efficient resources provisioning. RevPred in SpotTune improves the prediction accuracy by enhancing the data preprocessing approach and the model structure.

D. ML Training Prediction

Several systems predict the ML training process by either judging the model quality using a single-step metric [33] or fitting the training curve using a single function [18], [19]. They are suitable for limited scenarios because they miss the issue that the learning rate may have multiple stages. EarlyCurve in SpotTune uses staged function to handle this issue to reduce the prediction error.

VII. CONCLUSION

We present SpotTune, an orchestrating system leveraging transient resources in the public cloud for hyper-parameter tuning. SpotTune uses a fine-grained measurement *step cost* to choose the cheapest instance to run a single step, thus reducing the overall cost and increase the performance-cost rate. SpotTune also applies an ML training trend predicting model to early shutdown the training process to save computation capacity. In the experiments, we prove that using SpotTune in the public cloud could make HPT both fast and cheap. SpotTune also has some limitations. In our future work, we will consider enhancing SpotTune's support for checkpointing large models.

ACKNOWLEDGMENTS

This work is supported by the National Key R&D Program of China (2016YFB1000105), the Science Fund for Creative Research Groups of China under Grant, No. 61421091. We also thank our shepherd, Purushottam Kulkarni, for help in shaping the final version of the paper.

REFERENCES

- [1] A. Harlap, A. Chung, A. Tumanov, G. R. Ganger, and P. B. Gibbons, "Tributary: spot-dancing for elastic services with latency slo's," in *2018 {USENIX} Annual Technical Conference ({USENIX}{ATC} 18)*, 2018, pp. 1–14.
- [2] A. Harlap, A. Tumanov, A. Chung, G. R. Ganger, and P. B. Gibbons, "Proteus: agile ml elasticity through tiered reliability in dynamic resource markets," in *Proceedings of the Twelfth European Conference on Computer Systems*. ACM, 2017, pp. 589–604.
- [3] "AWS EC2 Spot Instances," <https://aws.amazon.com/ec2/spot/>, accessed Jan 9, 2020.
- [4] "Google Compute Engine," <https://cloud.google.com/compute/>, accessed Jan 9, 2020.
- [5] "AWS EC2," <https://aws.amazon.com/ec2/>, accessed Jan 9, 2020.
- [6] "AWS EC2 Spot Instances FAQs," <https://aws.amazon.com/ec2/faqs/#spot-instances>, accessed Jan 9, 2020.
- [7] "AWS EC2 Spot Instances Details," <https://aws.amazon.com/cn/ec2/spot/details/>, accessed Jan 9, 2020.
- [8] L. Zheng, C. Joe-Wong, C. W. Tan, M. Chiang, and X. Wang, "How to bid the cloud," in *ACM SIGCOMM Computer Communication Review*, vol. 45, no. 4. ACM, 2015, pp. 71–84.
- [9] P. Sharma, D. Irwin, and P. Shenoy, "Portfolio-driven resource management for transient cloud servers," *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, vol. 1, no. 1, p. 5, 2017.
- [10] S. Subramanya, T. Guo, P. Sharma, D. Irwin, and P. Shenoy, "Spoton: a batch computing service for the spot market," in *Proceedings of the sixth ACM symposium on cloud computing*. ACM, 2015, pp. 329–341.
- [11] G. Luo, "A review of automatic selection methods for machine learning algorithms and hyper-parameter values," *Network Modeling Analysis in Health Informatics and Bioinformatics*, vol. 5, no. 1, p. 18, 2016.
- [12] Z. Lu, C.-K. Chiang, and F. Sha, "Hyper-parameter tuning under a budget constraint," *arXiv preprint arXiv:1902.00532*, 2019.
- [13] B. Shahriari, K. Swersky, Z. Wang, R. P. Adams, and N. De Freitas, "Taking the human out of the loop: A review of bayesian optimization," *Proceedings of the IEEE*, vol. 104, no. 1, pp. 148–175, 2015.
- [14] J. Snoek, H. Larochelle, and R. P. Adams, "Practical bayesian optimization of machine learning algorithms," in *Advances in neural information processing systems*, 2012, pp. 2951–2959.
- [15] J. Bergstra and Y. Bengio, "Random search for hyper-parameter optimization," *Journal of Machine Learning Research*, vol. 13, no. Feb, pp. 281–305, 2012.
- [16] R. G. Mantovani, A. L. Rossi, J. Vanschoren, B. Bischl, and A. C. De Carvalho, "Effectiveness of random search in svm hyper-parameter tuning," in *2015 International Joint Conference on Neural Networks (IJCNN)*. Ieee, 2015, pp. 1–8.
- [17] E. R. Sparks, A. Talwalkar, D. Haas, M. J. Franklin, M. I. Jordan, and T. Kraska, "Automating model search for large scale machine learning," in *Proceedings of the Sixth ACM Symposium on Cloud Computing*. ACM, 2015, pp. 368–380.
- [18] Y. Peng, Y. Bao, Y. Chen, C. Wu, and C. Guo, "Optimus: an efficient dynamic resource scheduler for deep learning clusters," in *Proceedings of the Thirteenth EuroSys Conference*. ACM, 2018, p. 3.
- [19] H. Zhang, L. Stafman, A. Or, and M. J. Freedman, "Slaq: quality-driven scheduling for distributed machine learning," in *Proceedings of the 2017 Symposium on Cloud Computing*. ACM, 2017, pp. 390–404.
- [20] "Epsilon Dataset," <http://largescale.ml.tu-berlin.de/instructions/>, accessed Jan 9, 2020.
- [21] "Yearpredictionmsd dataset," <https://archive.ics.uci.edu/ml/datasets/yearpredictionmsd>, accessed Jan 9, 2020.
- [22] "Cifar10 Dataset," <https://www.cs.toronto.edu/~kriz/cifar.html>, accessed Jan 9, 2020.
- [23] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.
- [24] "Python Least Square Solver," https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.least_squares.html, accessed Jan 9, 2020.
- [25] B. Visser, "AWS Spot Pricing Market," <https://www.kaggle.com/noqcks/aws-spot-pricing-market>, accessed Jan 9, 2020.
- [26] "MNIST Dataset," <http://yann.lecun.com/exdb/mnist/>, accessed Jan 9, 2020.
- [27] "FUSE-based file system backed by Amazon S3," <https://github.com/s3fs-fuse/s3fs-fuse>, accessed Jan 9, 2020.
- [28] "AWS Simple Storage Service(S3)," <https://docs.aws.amazon.com/s3/index.html>, accessed Jan 9, 2020.
- [29] P. Joaquim, M. Bravo, L. Rodrigues, and M. Matos, "Hourglass: Leveraging transient resources for time-constrained graph processing in the cloud," in *Proceedings of the Fourteenth EuroSys Conference 2019*. ACM, 2019, p. 35.
- [30] S. Shastri and D. Irwin, "Hotspot: automated server hopping in cloud spot markets," in *Proceedings of the 2017 Symposium on Cloud Computing*. ACM, 2017, pp. 493–505.
- [31] A. Ali-Eldin, J. Westin, B. Wang, P. Sharma, and P. Shenoy, "Spotweb: Running latency-sensitive distributed web services on transient cloud servers," *HPDC*, June, 2019.
- [32] C. Zhang, M. Yu, W. Wang, and F. Yan, "Mark: Exploiting cloud services for cost-effective, slo-aware machine learning inference serving," in *2019 {USENIX} Annual Technical Conference ({USENIX}{ATC} 19)*, 2019, pp. 1049–1062.
- [33] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, "Hyperband: A novel bandit-based approach to hyperparameter optimization," *arXiv preprint arXiv:1603.06560*, 2016.